

Apache Kafka for Beginners

A theory-first, visually structured walkthrough of Kafka's core concepts — topics, partitions, brokers, producers, consumers, and delivery guarantees — built for interview preparation and real-world understanding.

Distributed Systems

Event Streaming

Interview-Ready

Table of Contents

01	What Is Apache Kafka and Why It Exists
02	Core Building Blocks: Topics, Partitions & Offsets
03	Producers — Writing Data Into Kafka
04	Consumers & Consumer Groups
05	Brokers, Clusters & Replication
06	Delivery Guarantees & Message Semantics
07	ZooKeeper vs. KRaft — How Kafka Manages Itself
08	Kafka vs. Traditional Message Queues
09	Real-World Use Cases
10	Essential CLI Commands Cheat Sheet
11	Interview Quick-Recap

What Is Apache Kafka and Why It Exists

Apache Kafka is a **distributed event streaming platform**. In plain terms, it is a system designed to move large volumes of data — continuously and in real time — from the places that produce it to the places that need to use it, reliably and at scale.

Kafka was originally built at LinkedIn in 2011 to solve a very concrete problem: the company had dozens of systems that needed to share data with each other — user activity, logs, metrics, database changes — and connecting every system directly to every other system created an unmanageable web of point-to-point pipelines. Kafka was designed to sit in the middle as a single, durable, high-throughput backbone that all systems could publish to and read from.

ANALOGY

Think of Kafka as a **central postal system** for your company's data. Instead of every department mailing letters directly to every other department (chaotic, hard to track, easy to lose mail), everyone drops mail into one well-organized post office. The post office stores it safely, keeps a permanent record, and lets any department pick up exactly the mail that's relevant to them — whenever they're ready.

The Problem Kafka Solves

Before systems like Kafka, companies typically faced two options, both flawed at scale:

- **Point-to-point integration** — every producer of data connects directly to every consumer of that data. This grows quadratically: N systems require roughly N^2 connections, each with its own format, retry logic, and failure mode.
- **A single shared database** — every system writes to and reads from one central database. This becomes a bottleneck, a single point of failure, and forces every system to agree on one schema and one technology.

Kafka replaces both patterns with a **publish–subscribe log**: producers write data once, Kafka stores it durably, and any number of independent consumers can read it independently, at their own pace, without producers ever knowing who is listening.



What Makes Kafka Different From a Normal Queue

Traditional message queues (like older versions of RabbitMQ or ActiveMQ) typically **delete a message once it has been consumed**. Kafka does the opposite: it stores every message on disk for a configurable retention period (hours, days, or forever), regardless of whether anyone has read it yet. This single design decision is the root of almost everything that makes Kafka distinctive:

- Multiple independent consumers can read the exact same data, each at their own pace.
- A new consumer can be added later and still read historical data from the beginning.
- Consumers can "rewind" and reprocess data if a bug is found downstream.
- Kafka behaves less like a mailbox and more like an **append-only, replayable log**.

KEY MENTAL MODEL

Kafka is not primarily a "messaging system" — it is a **distributed, durable, append-only commit log** that happens to support publish/subscribe messaging patterns on top of that log.

Where Kafka Fits in a Data Architecture

Kafka commonly sits between operational systems (applications, databases, IoT devices, microservices) and analytical or downstream systems (data warehouses, data lakes, dashboards, fraud-detection engines, search indexes). It decouples the two worlds: producers don't need to know what will eventually consume their data, and consumers don't need to know who produced it.

Without Kafka	With Kafka
Every system connects directly to every other system	Every system connects to one Kafka cluster
Tight coupling between producers and consumers	Producers and consumers are fully decoupled
Data is processed once and often discarded	Data is retained and can be replayed
Hard to add a new downstream system	New consumers can be added with zero impact on producers

Chapter Summary

- Kafka is a distributed event streaming platform built around a durable, append-only log.
- It decouples data producers from data consumers.
- Unlike traditional queues, Kafka retains messages after they've been read, enabling replay.

- It was created at LinkedIn to solve the N² integration problem between internal systems.

Core Building Blocks: Topics, Partitions & Offsets

Topics — The Named Category of Data

A **topic** is the fundamental unit of organization in Kafka — essentially a named stream of related data, similar to a table in a database or a folder in a file system. Examples might be `orders`, `page_views`, or `payment_events`. Producers write to topics, and consumers read from topics.

Topics are **logical** — the actual data isn't stored as one single file, it is split across one or more **partitions**, which is where Kafka's scalability comes from.

Partitions — How Kafka Scales

Each topic is divided into one or more partitions, and each partition is an independent, strictly **ordered, append-only log**. When a producer sends a message, it is appended to the end of one partition and given a permanent, sequential ID within that partition called an **offset**.

```
TOPIC: orders (3 partitions)
Partition 0: [msg0][msg1][msg2][msg3][msg4] ...
Partition 1: [msg0][msg1][msg2] ...
Partition 2: [msg0][msg1][msg2][msg3] ...
# Each partition is its own ordered, append-only sequence
```

Partitioning is what allows Kafka to scale horizontally: partitions of a topic can be spread across multiple brokers (servers) in a cluster, and multiple consumers can read different partitions in parallel. More partitions generally means more parallelism, at the cost of more overhead and more complex ordering guarantees.

WARNING — ORDERING IS PER-PARTITION, NOT PER-TOPIC

Kafka only guarantees message order **within a single partition** — never across an entire topic. If strict ordering matters for a given entity (e.g. all events for one customer), that entity's messages must always be routed to the same partition.

Offsets — A Message's Address

An **offset** is a simple, ever-increasing integer that identifies a message's position within its partition. Offsets are unique within a partition, but not across the whole topic — partition 0's offset 5 and partition 1's offset 5 are two completely different messages.

Consumers track which offset they have last read, which is how Kafka lets a consumer stop, crash, restart, or pause, and later resume exactly where it left off.

Keys and Partition Assignment

Every message in Kafka can optionally carry a **key**. If a key is provided, Kafka runs it through a hash function to consistently route all messages with that same key to the **same partition** — which is exactly how ordering per-entity is preserved. If no key is provided, messages are distributed round-robin (or via a "sticky" partitioner) across partitions for load balancing.

Concept	What It Is	Analogy
Topic	A named stream/category of data	A book title
Partition	An ordered, append-only slice of a topic	A single chapter
Offset	A message's position within a partition	A page number in that chapter
Key	Determines which partition a message lands in	A label saying "put this in chapter 2"

INTERVIEW QUESTION

"How does Kafka guarantee message ordering?" — Answer: Kafka guarantees order only within a single partition. To keep related events in order, use a consistent partition key (e.g. a customer ID or order ID) so all related messages land on the same partition.

Chapter Summary

- Topics are logical categories of data; partitions are the physical, ordered logs behind them.
- Offsets uniquely identify a message's position within its partition.
- Ordering is only guaranteed within a partition — use keys to keep related data together.
- More partitions = more parallelism, but ordering guarantees only apply per-partition.

Producers — Writing Data Into Kafka

A **producer** is any client application that publishes (writes) messages to Kafka topics. Producers are intentionally "dumb" from Kafka's point of view — they don't need to know who, if anyone, is consuming the data.

The Producer Workflow

1. The application creates a message, optionally with a key and always with a value.
2. The producer's **partitioner** decides which partition the message goes to (based on the key, or round-robin if there's no key).
3. The message is batched with others headed to the same partition/broker for efficiency.
4. The batch is sent to the partition's **leader broker**.
5. Depending on the configured `acks` setting, the producer waits for confirmation.

The `acks` Setting — Trading Off Speed vs. Safety

This is one of the most commonly asked Kafka configuration questions, because it directly controls the durability guarantee a producer gets.

acks value	Behavior	Trade-off
<code>acks=0</code>	Producer doesn't wait for any confirmation	Fastest, but messages can be silently lost
<code>acks=1</code>	Waits for the partition leader to acknowledge	Balanced — safe unless the leader fails before replicating
<code>acks=all</code> (or <code>-1</code>)	Waits for all in-sync replicas to acknowledge	Strongest durability, higher latency

CONCEPTUAL PRODUCER CONFIG

```
bootstrap.servers = broker1:9092,broker2:9092
acks = all
retries = 3
key.serializer = StringSerializer
value.serializer = JsonSerializer
# acks=all is the standard choice when data loss is unacceptable
```


Batching and Compression

To achieve high throughput, producers don't send one message at a time — they group messages destined for the same partition into batches, controlled by settings like `batch.size` and `linger.ms` (how long to wait for a batch to fill before sending anyway). Batches are commonly compressed (e.g. `snappy` , `gzip` , `lz4`) to reduce network and disk usage.

TIP — THROUGHPUT VS. LATENCY

Increasing `linger.ms` slightly (e.g. 5–20ms) lets more messages accumulate per batch, dramatically improving throughput at the cost of a small, usually imperceptible delay. This is a classic Kafka tuning trade-off worth knowing for interviews.

Chapter Summary

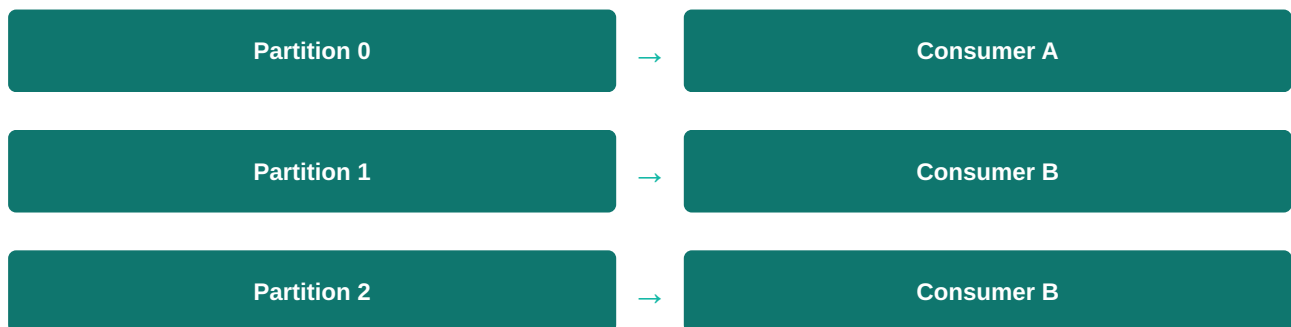
- Producers write messages to topics without needing to know who will consume them.
- The `acks` setting controls the durability/speed trade-off.
- Producers batch and compress messages to maximize throughput.

Consumers & Consumer Groups

A **consumer** reads messages from one or more partitions. Consumers pull data from Kafka (rather than Kafka pushing it to them), which lets each consumer control its own pace of processing.

Consumer Groups

Consumers are almost always organized into **consumer groups** — a named set of consumer instances that cooperate to read a topic. Kafka guarantees that **each partition is read by only one consumer within a given group at any time**, which is how Kafka achieves parallel processing without duplicate work inside a group.



Example: a 3-partition topic read by a 2-consumer group — Consumer B reads two partitions.

Why Partition Count Limits Parallelism

If a consumer group has **more consumers than partitions**, the extra consumers will sit idle — a partition can't be split further between two consumers in the same group. This is why the number of partitions on a topic sets a hard ceiling on how much you can parallelize consumption within one group.

WARNING — COMMON INTERVIEW TRAP

"If I have 10 partitions and 15 consumers in one group, what happens?" — 5 consumers will remain completely idle, because Kafka never assigns more than one consumer per partition within the same group.

Multiple Groups Reading the Same Topic

Different consumer groups are fully independent of each other. Two separate applications (e.g. a fraud-detection service and an analytics pipeline) can both read the exact same topic from the beginning, at their own pace, without interfering with one another — each group tracks its own offsets.

Rebalancing

When a consumer joins or leaves a group (including crashing), Kafka triggers a **rebalance**: partitions are reassigned among the remaining/new consumers. Rebalances are necessary for fault tolerance and elasticity, but they briefly pause consumption for the group, so overly frequent rebalances (e.g. from flaky consumers) can hurt throughput.

Committing Offsets

Consumers periodically **commit** the offset of the last message they've successfully processed, so that if they restart, they know where to resume. This can be done automatically (`enable.auto.commit=true`) or manually for tighter control over exactly-once-style processing.

Chapter Summary

- Consumer groups let multiple consumer instances share the work of reading a topic.
- Each partition is consumed by exactly one consumer per group at a time.
- Different groups are independent and can each read the full topic on their own timeline.
- Rebalancing redistributes partitions when group membership changes.

Brokers, Clusters & Replication

Brokers

A **broker** is a single Kafka server — it stores data, serves producer writes and consumer reads, and coordinates with other brokers. A **cluster** is a group of brokers working together, typically identified by a unique broker ID within the cluster.

Each broker hosts some subset of the partitions across all topics. As a cluster grows, partitions and their replicas are distributed across more brokers to spread out load and storage.

Replication — Kafka's Fault Tolerance Mechanism

Every partition can be configured with a **replication factor** — the number of copies of that partition maintained across different brokers. One replica is elected the **leader**; all reads and writes for that partition go through the leader, while the other replicas (**followers**) continuously copy data from the leader to stay in sync.

```
TOPIC "orders" · PARTITION 0 · REPLICATION FACTOR 3 Broker 1: [LEADER] ←
handles all reads/writes for this partition
Broker 2: [FOLLOWER] ← replicates from the leader
Broker 3: [FOLLOWER] ← replicates from the leader
# If Broker 1 fails, one of the followers is promoted to leader automatically
```

In-Sync Replicas (ISR)

The **ISR** is the set of replicas that are fully caught up with the leader at any given moment. Only replicas in the ISR are eligible to be promoted to leader if the current leader fails — this prevents Kafka from electing a replica that's missing recent data and silently losing messages.

TIP — `min.insync.replicas`

Combined with `acks=all`, the topic-level setting `min.insync.replicas` defines how many replicas must acknowledge a write before it's considered successful. Setting this to 2 with a replication factor of 3 means the cluster can tolerate one broker failure with zero data loss.

Leader Election and Failover

If a leader broker fails, the Kafka controller (a broker elected to manage cluster metadata) promotes one of the in-sync followers to become the new leader — this typically happens within seconds, and producers/consumers automatically discover the new leader and redirect traffic.

Why Replication Factor Matters

Replication Factor	Broker Failures Tolerated	Typical Use
1	0 (any broker loss = data loss)	Local dev / testing only
2	1	Cost-sensitive, moderate risk tolerance
3	2	Standard production default

Chapter Summary

- Brokers are individual Kafka servers; a cluster is a set of cooperating brokers.
- Each partition has one leader and zero or more follower replicas.
- The ISR tracks which replicas are safe to promote to leader on failure.
- Replication factor 3 with `min.insync.replicas=2` is a common production standard.

Delivery Guarantees & Message Semantics

One of the most frequently tested Kafka topics in interviews is delivery semantics — what guarantees Kafka can make about whether a message is delivered **zero, one, or possibly more than once**.

Semantic	Meaning	Risk
At most once	Message may be lost, but never processed twice	Data loss possible
At least once	Message is never lost, but may be processed more than once	Duplicate processing possible
Exactly once	Message is delivered and processed exactly one time	Hardest to achieve; some performance cost

At Most Once

Occurs when a consumer commits its offset **before** processing the message. If the consumer crashes after committing but before finishing processing, that message is effectively skipped on restart.

At Least Once (the common default)

Occurs when a consumer commits its offset **after** processing the message. If the consumer crashes after processing but before committing, it will reprocess that message on restart — safe, but requires downstream systems to be able to handle duplicates (idempotent processing).

Exactly Once Semantics (EOS)

Kafka supports exactly-once semantics through a combination of an **idempotent producer** (which avoids duplicate writes on retries using sequence numbers) and **transactions** (which let a producer atomically write to multiple partitions and commit consumer offsets together, so either everything succeeds or nothing does).

```
PRODUCER CONFIG FOR EXACTLY-ONCE enable.idempotence = true
transactional.id = "order-processor-1"
acks = all
# Idempotence + transactions together enable exactly-once writes
```

INTERVIEW QUESTION

"Is Kafka exactly-once by default?" — No. The default and most common configuration is at-least-once. Exactly-once requires explicitly enabling idempotent producers and transactions, and is typically scoped to Kafka-to-Kafka pipelines (e.g. Kafka Streams).

Chapter Summary

- Delivery semantics describe how many times a message might be processed.
- At-least-once is the most common default; it requires idempotent consumers.
- Exactly-once relies on idempotent producers plus Kafka transactions.

ZooKeeper vs. KRaft — How Kafka Manages Itself

Kafka needs to track cluster-wide metadata: which brokers exist, which broker leads which partition, topic configurations, and access control. Historically this was handled by an external system; more recently Kafka has moved to managing it internally.

The Old Model: Apache ZooKeeper

For most of Kafka's history, a separate distributed coordination service called **ZooKeeper** stored cluster metadata and handled controller election. This worked, but required operating and scaling an entirely separate system alongside Kafka itself, and became a scalability bottleneck for clusters with very large numbers of partitions.

The New Model: KRaft (Kafka Raft)

KRaft removes the ZooKeeper dependency entirely by having a subset of Kafka brokers themselves (acting as "controllers") manage metadata using the Raft consensus algorithm. This simplifies deployment (one system instead of two), improves metadata scalability, and speeds up controller failover.

Aspect	ZooKeeper Mode	KRaft Mode
External dependency	Yes — separate ZooKeeper ensemble	No — metadata managed inside Kafka
Metadata scalability	Limited at very large partition counts	Significantly higher
Operational complexity	Two systems to run and monitor	One system
Status	Deprecated / being phased out	Default for new Kafka clusters

TIP — WHAT TO REMEMBER

You don't need deep Raft internals for most interviews — the key point is that **KRaft is Kafka's newer, built-in replacement for ZooKeeper** that removes an external dependency and improves scalability.

Chapter Summary

- ZooKeeper was historically used to manage Kafka cluster metadata and controller election.
- KRaft replaces ZooKeeper with Kafka-internal, Raft-based consensus.
- KRaft is now the default and recommended mode for new Kafka deployments.

Kafka vs. Traditional Message Queues

Kafka is often compared to traditional message brokers like RabbitMQ or ActiveMQ. They solve overlapping problems, but their underlying models are quite different — a common interview topic.

Aspect	Traditional Queue (e.g. RabbitMQ)	Apache Kafka
Data model	Queue — message removed once consumed	Log — message retained after being read
Multiple consumers	Message typically goes to only one consumer	Many independent consumer groups can all read everything
Replay	Generally not supported	Native — rewind to any offset
Throughput	Good, optimized for complex routing	Very high, optimized for sequential log writes
Message ordering	Per-queue, often global	Per-partition only
Best fit	Complex routing, task queues, RPC-style messaging	High-volume event streaming, log aggregation, real-time pipelines

INTERVIEW QUESTION

"When would you choose RabbitMQ over Kafka?" — When you need complex routing logic (exchanges, priority queues), lower total throughput requirements, or a strict one-message-to-one-consumer task-queue pattern rather than a replayable event stream.

Chapter Summary

- Traditional queues delete messages after consumption; Kafka retains them as a log.
- Kafka is built for very high throughput and multiple independent readers.
- Traditional brokers often win for complex routing and classic task-queue patterns.

Real-World Use Cases

Log and Event Aggregation

Centralizing application logs, metrics, and events from hundreds of microservices into a single durable stream that monitoring, alerting, and analytics systems can all subscribe to.

Real-Time Data Pipelines (CDC)

Kafka is a common backbone for **Change Data Capture** pipelines, where changes in a source database (inserts, updates, deletes) are streamed into Kafka in real time and then consumed by data warehouses, search indexes, or caches to keep them synchronized.

Stream Processing

Tools like **Kafka Streams** or Apache Flink read from Kafka topics, perform real-time transformations, aggregations, or joins, and write results back to other Kafka topics — enabling continuous, low-latency analytics rather than periodic batch jobs.

Activity Tracking

Capturing user activity on websites and apps (clicks, page views, searches) as a high-throughput stream that feeds recommendation engines, analytics dashboards, and personalization systems.

Microservices Communication (Event-Driven Architecture)

Services publish domain events (e.g. `OrderPlaced`, `PaymentProcessed`) to Kafka rather than calling each other directly, decoupling services and allowing them to evolve, scale, and fail independently.

Metrics and Monitoring

Aggregating operational metrics from distributed infrastructure into Kafka before they're processed and visualized by monitoring platforms.

TIP — COMPANIES USING KAFKA AT SCALE

LinkedIn (its birthplace), Netflix, Uber, Airbnb, and thousands of other companies run Kafka to handle trillions of messages per day across logging, real-time recommendations, fraud detection, and operational monitoring.

Chapter Summary

- Kafka underpins log aggregation, CDC pipelines, stream processing, and event-driven microservices.
- Its durability and replay capability make it ideal wherever multiple systems need the same data.

Essential CLI Commands Cheat Sheet

While this guide is theory-focused, it helps to recognize what interacting with Kafka actually looks like at the command line.

Topic Management

```
CREATE A TOPIC kafka-topics.sh --create \  
--topic orders \  
--bootstrap-server localhost:9092 \  
--partitions 3 \  
--replication-factor 3
```

```
LIST TOPICS kafka-topics.sh --list --bootstrap-server localhost:9092
```

```
DESCRIBE A TOPIC kafka-topics.sh --describe \  
--topic orders \  
--bootstrap-server localhost:9092  
# Shows partition count, leader, replicas, and ISR per partition
```

Producing & Consuming

```
PRODUCE MESSAGES FROM THE CONSOLE kafka-console-producer.sh \  
--topic orders \  
--bootstrap-server localhost:9092
```

```
CONSUME MESSAGES FROM THE BEGINNING kafka-console-consumer.sh \  
--topic orders \  
--from-beginning \  
--bootstrap-server localhost:9092
```

Consumer Group Inspection

```
LIST CONSUMER GROUPS kafka-consumer-groups.sh --list --bootstrap-server  
localhost:9092
```

```
CHECK CONSUMER LAG kafka-consumer-groups.sh --describe \  
--group order-processor-group \  
--bootstrap-server localhost:9092  
# LAG column shows how far behind each partition's consumption is
```

TIP — CONSUMER LAG

Lag = (latest offset in the partition) – (last committed offset for the consumer). It's the single most important health metric for a Kafka pipeline — rising lag means consumers can't keep up with producers.

Chapter Summary

- kafka-topics.sh manages topic creation and inspection.
- kafka-console-producer.sh / kafka-console-consumer.sh are quick manual testing tools.
- kafka-consumer-groups.sh reveals group membership and, critically, consumer lag.

Interview Quick-Recap

Rapid-Fire Definitions

Term	One-Line Definition
Topic	A named stream of related messages
Partition	An ordered, append-only log within a topic
Offset	A message's position within its partition
Broker	A single Kafka server
Cluster	A group of brokers working together
Producer	A client that writes messages to topics
Consumer	A client that reads messages from topics
Consumer Group	A set of consumers sharing the work of reading a topic
Replication Factor	How many copies of each partition exist across brokers
ISR	Replicas fully caught up with the partition leader
Leader / Follower	The replica handling reads/writes vs. replicas copying it
Consumer Lag	How far behind a consumer is from the latest message
KRaft	Kafka's built-in, ZooKeeper-free metadata management

Questions Worth Rehearsing Out Loud

- Why does Kafka only guarantee ordering within a partition, not across a topic?
- Walk through what happens when a broker holding a partition leader crashes.
- What's the difference between at-least-once and exactly-once delivery, and how does Kafka achieve each?
- What happens if a consumer group has more consumers than the topic has partitions?
- Why would you choose Kafka over a traditional message queue — and when wouldn't you?

- What is consumer lag, and why does it matter operationally?

FINAL TIP

Almost every Kafka interview question traces back to one of three ideas: the **partitioned log** model, the **leader/follower replication** model, and the **at-least-once by default** delivery model. If you can explain those three clearly with an example, you can reason through nearly any follow-up question.

— End of Guide —